

Add maps & real-time location to SparkClean

using Leaflet.js + OpenStreetMap.

100% free, no API keys needed.

TECH STACK

Map display : Leaflet.js + react-leaflet

Map tiles : CartoDB Dark (matches #0A0A0A theme)

Address search: Photon API (free, no key)

ETA routing : OSRM API (free, no key)

Live tracking : Socket.io (already on backend)

INSTALL

```
npm install leaflet react-leaflet
```

```
npm install @types/leaflet
```

```
npm install socket.io-client
```

Backend:

```
npm install socket.io
```

No .env changes needed — zero API keys!

STEP 1 — Fix Leaflet CSS in index.tsx

Open src/index.tsx

ADD at very top:

```
import 'leaflet/dist/leaflet.css'
```

Also fix Leaflet default icon bug.

Open src/index.tsx ADD:

```
import L from 'leaflet'

import markerIcon from
  'leaflet/dist/images/marker-icon.png'

import markerIcon2x from
  'leaflet/dist/images/marker-icon-2x.png'

import markerShadow from
  'leaflet/dist/images/marker-shadow.png'
```

```
delete (L.Icon.Default.prototype as any)
  ._getIconUrl
```

```
L.Icon.Default.mergeOptions({
  iconUrl    : markerIcon,
  iconRetinaUrl: markerIcon2x,
  shadowUrl  : markerShadow,
})
```

STEP 2 — Custom Map Icons

CREATE src/lib/mapIcons.ts

```
import L from 'leaflet'

export const tealPin = new L.DivIcon({
  html: `
    <div style="
      width    : 32px;
```

```

    height    : 32px;
    background : #0AFFE6;
    border-radius: 50% 50% 50% 0;
    transform  : rotate(-45deg);
    border     : 3px solid #000;
    box-shadow : 0 2px 8px rgba(10,255,230,0.5);
"></div>
`,
iconSize : [32, 32],
iconAnchor : [16, 32],
popupAnchor: [0, -32],
className : "",
})

```

```

export const homePin = new L.DivIcon({
  html: `
<div style="
  width      : 36px;
  height     : 36px;
  background  : #FFFFFF;
  border-radius : 50%;
  border      : 3px solid #0AFFE6;
  display     : flex;
  align-items : center;
  justify-content : center;
  font-size   : 18px;
  box-shadow  : 0 2px 8px rgba(0,0,0,0.3);
">🏠</div>
`,
  iconSize : [36, 36],
  iconAnchor : [18, 18],

```

```

    popupAnchor: [0, -18],
    className : "",
  })

export const cleanerPin = new L.DivIcon({
  html: `
    <div style="
      width      : 40px;
      height     : 40px;
      background  : #0AFFE6;
      border-radius : 50%;
      border     : 3px solid #000;
      display    : flex;
      align-items : center;
      justify-content : center;
      font-size   : 20px;
      box-shadow  : 0 2px 12px rgba(10,255,230,0.6);
      animation   : pulse 2s infinite;
    "> 🧹 </div>
  `,
  iconSize : [40, 40],
  iconAnchor : [20, 20],
  popupAnchor: [0, -20],
  className : "",
})

```

STEP 3 — Dark Map Tile Config

CREATE src/lib/mapConfig.ts

```
export const DARK_TILES = {  
  url      : 'https://{s}.basemaps.cartocdn.com/dark_all/{z}/{x}/{y}{r}.png',  
  attribution: '&copy; OpenStreetMap &copy; CartoDB',  
}
```

```
export const LIGHT_TILES = {  
  url      : 'https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png',  
  attribution: '&copy; OpenStreetMap contributors',  
}
```

```
export const CITY_CENTERS = {  
  Bengaluru: { lat: 12.9716, lng: 77.5946 },  
  Mumbai   : { lat: 19.0760, lng: 72.8777 },  
  India    : { lat: 20.5937, lng: 78.9629 },  
}
```

STEP 4 — Address Autocomplete Component

CREATE src/components/AddressSearch.tsx

Uses Photon API — no key needed:

```
import { useState, useEffect, useRef } from 'react'
```

```
interface Suggestion {
```

```
  label: string
```

```
  lat  : number
```

```
  lng  : number
```

```
}
```

```
interface Props {  
  value : string  
  onChange : (val: string) => void  
  onSelect : (data: {  
    address: string  
    lat : number  
    lng : number  
  }) => void  
  placeholder?: string  
}
```

```
const AddressSearch = ({  
  value, onChange, onSelect,  
  placeholder = 'Search your address...'  
}: Props) => {  
  const [suggestions, setSuggestions] =  
    useState<Suggestion[]>([])  
  const [loading, setLoading] =  
    useState(false)  
  const [showList, setShowList] =  
    useState(false)  
  const timerRef = useRef<any>(null)
```

```
  useEffect(() => {  
    if (value.length < 3) {  
      setSuggestions([])  
      return  
    }  
  })
```

```
clearTimeout(timerRef.current)

timerRef.current = setTimeout(async () => {
  setLoading(true)

  try {
    const url =
      `https://photon.komoot.io/api/?` +
      `q=${encodeURIComponent(value+' India')}` +
      `&limit=6&lang=en`

    const res = await fetch(url)
    const data = await res.json()

    const results = data.features.map(
      (f: any) => ({
        label: [
          f.properties.name,
          f.properties.street,
          f.properties.city,
          f.properties.state,
        ].filter(Boolean).join(', '),
        lat: f.geometry.coordinates[1],
        lng: f.geometry.coordinates[0],
      })
    )

    setSuggestions(results)
    setShowList(true)
  } catch (err) {
    console.error('Address search failed:', err)
  } finally {
    setLoading(false)
  }
}
```

```
}, 400)
```

```
return () => clearTimeout(timerRef.current)  
}, [value])
```

```
return (  
  <div style={{ position:'relative' }}>  
    <div style={{ position:'relative' }}>  
      <input  
        value={value}  
        onChange={e => onChange(e.target.value)}  
        onFocus={() => setShowList(true)}  
        onBlur={() =>  
          setTimeout(() => setShowList(false), 200)  
        }  
        placeholder={placeholder}  
        style={{  
          width      : '100%',  
          padding    : '14px 44px 14px 16px',  
          background : 'white',  
          border     : '2px solid',  
          borderColor : showList  
            ? '#0AFFE6' : '#E2E8F0',  
          borderRadius : '12px',  
          fontSize   : '15px',  
          outline    : 'none',  
          transition : 'border-color 0.2s',  
          boxSizing  : 'border-box',  
        }}  
      />  
      <span style={{
```

```

position : 'absolute',
right    : '14px',
top      : '50%',
transform : 'translateY(-50%)',
fontSize : '18px',
}}>
{loading ? '⌚' : '📍'}
</span>
</div>

```

```

{showList && suggestions.length > 0 && (
  <div style={{
    position    : 'absolute',
    top         : 'calc(100% + 4px)',
    left        : 0,
    right       : 0,
    background   : 'white',
    border      : '1px solid #E2E8F0',
    borderRadius : '12px',
    boxShadow    : '0 8px 24px rgba(0,0,0,0.12)',
    zIndex      : 9999,
    maxHeight    : '280px',
    overflowY    : 'auto',
  }}>
    {suggestions.map((s, i) => (
      <div
        key={i}
        onMouseDown={() => {
          onSelect({
            address: s.label,
            lat    : s.lat,

```

```

      lng : s.lng,
    })
    onChange(s.label)
    setSuggestions([])
    setShowList(false)
  })
  style={{
    padding : '12px 16px',
    cursor : 'pointer',
    fontSize : '14px',
    color : '#1A1A1A',
    borderBottom : i < suggestions.length - 1
      ? '1px solid #F5F5F5' : 'none',
    display : 'flex',
    alignItems : 'flex-start',
    gap : '10px',
    transition : 'background 0.1s',
  }}
  onMouseEnter={e =>
    (e.currentTarget.style.background = '#F0FFFE')
  }
  onMouseLeave={e =>
    (e.currentTarget.style.background = 'white')
  }
>
<span style={{
  marginTop : '1px',
  flexShrink: 0,
  fontSize : '16px'
}}> 📌 </span>
<span style={{ lineHeight: '1.4' }}>

```

```

        {s.label}
      </span>
    </div>
  )))
</div>
})
</div>
)
}

```

export default AddressSearch

STEP 5 — Update BookingPage.tsx

In BookingPage.tsx:

ADD state for coordinates:

```

const [coords, setCoords] = useState<{
  lat: number | null
  lng: number | null
}>({ lat: null, lng: null })

```

REPLACE plain address input with:

```

import AddressSearch from
  '../components/AddressSearch'

```

```

<AddressSearch
  value={formData.address}
  onChange={(val) =>

```

```

      setFormData({...formData, address: val})
    }
    onSelect={({ address, lat, lng }) => {
      setFormData({...formData, address})
      setCoords({ lat, lng })
    }}
    placeholder="House/flat no., street, landmark..."
  />

```

ADD to booking submit data:

```

addressLat: coords.lat,
addressLng: coords.lng,

```

STEP 6 — BookingMap Component

CREATE src/components/BookingMap.tsx

Shows pin at booked address on SuccessPage:

```

import { MapContainer, TileLayer,
  Marker, Popup } from 'react-leaflet'
import { tealPin } from '../lib/mapIcons'
import { DARK_TILES } from '../lib/mapConfig'

interface Props {
  lat : number
  lng : number
  address : string
}

```

```
const BookingMap = ({ lat, lng, address }: Props) => {  
  if (!lat || !lng) return null
```

```
  return (  
    <div style={{
```

```
      borderRadius : '16px',
```

```
      overflow      : 'hidden',
```

```
      border        : '1px solid rgba(10,255,230,0.2)',
```

```
      marginTop     : '20px',
```

```
    }}>
```

```
    <div style={{
```

```
      background : 'rgba(10,255,230,0.08)',
```

```
      padding    : '12px 16px',
```

```
      display    : 'flex',
```

```
      alignItems : 'center',
```

```
      gap        : '8px',
```

```
    }}>
```

```
      <span style={{
```

```
        fontSize:'18px' }}> 📍 </span>
```

```
      <span style={{
```

```
        color    : '#0AFFE6',
```

```
        fontSize : '14px',
```

```
        fontWeight:'600'
```

```
      }}>
```

```
        Your service location
```

```
      </span>
```

```
    </div>
```

```
  <MapContainer
```

```
    center={[lat, lng]}
```

```
    zoom={15}
```

```

style={{ height:'240px', width:'100%' }}
zoomControl={true}
scrollWheelZoom={false}
attributionControl={false}
>
<TileLayer
  url={DARK_TILES.url}
  attribution={DARK_TILES.attribution}
/>
<Marker position={[lat, lng]} icon={tealPin}>
  <Popup>
    <strong>Service here</strong><br/>
    {address}
  </Popup>
</Marker>
</MapContainer>

```

```

<div style={{
  background : '#0A0A0A',
  padding   : '10px 16px',
  display   : 'flex',
  alignItems : 'center',
  gap       : '6px',
}}>
  <span style={{ fontSize:'14px' }}> 📍 </span>
  <span style={{
    color : '#A0A0A0',
    fontSize : '13px',
  }}>
    {address}
  </span>

```

```

    </div>
  </div>
)
}

```

export default BookingMap

ADD to SuccessPage.tsx:

```
import BookingMap from '../components/BookingMap'
```

```

{booking?.addressLat && (
  <BookingMap
    lat={booking.addressLat}
    lng={booking.addressLng}
    address={booking.address}
  />
)}

```

STEP 7 — Live Tracking Map Component

CREATE src/components/LiveTrackingMap.tsx

Shows cleaner moving to customer in real time:

```

import { useEffect, useState, useRef } from 'react'
import { MapContainer, TileLayer,
  Marker, Polyline,
  useMap } from 'react-leaflet'
import { io, Socket } from 'socket.io-client'

```

```
import { homePin, cleanerPin } from '../lib/mapIcons'
import { DARK_TILES } from '../lib/mapConfig'
```

```
interface Position {
  lat: number
  lng: number
}
```

```
interface Props {
  bookingId : string
  customerLat: number
  customerLng: number
}
```

```
const MapPanner = ({ pos }: { pos: Position | null }) => {
  const map = useMap()
  useEffect(() => {
    if (pos) map.panTo([pos.lat, pos.lng],
      { animate: true, duration: 1 })
  }, [pos])
  return null
}
```

```
const LiveTrackingMap = ({
  bookingId, customerLat, customerLng
}: Props) => {
  const [cleanerPos, setCleanerPos] =
    useState<Position | null>(null)
  const [trail, setTrail] =
    useState<Position[]>([])
  const [eta, setEta] =
```

```
    useState<string | null>(null)
const [connected, setConnected ] =
    useState(false)
const [lastUpdate, setLastUpdate ] =
    useState<string | null>(null)
const socketRef = useRef<Socket | null>(null)
```

```
useEffect(() => {
    socketRef.current = io(
        process.env.REACT_APP_API_URL ||
        'http://localhost:3001'
    )
})
```

```
socketRef.current.on('connect', () => {
    setConnected(true)
    socketRef.current?.emit(
        'customer:watch', { bookingId }
    )
})
```

```
socketRef.current.on('disconnect', () => {
    setConnected(false)
})
```

```
socketRef.current.on(
    'location:update',
    async ({ lat, lng }: Position) => {
        setCleanerPos({ lat, lng })
        setTrail(prev => [
            ...prev.slice(-30),
            { lat, lng }
        ])
    })
})
```

```

    ])

    setLastUpdate(
      new Date().toLocaleTimeString('en-IN')
    )

    // Get ETA from OSRM (free)
    try {
      const url =
        `https://router.project-osrm.org/route/v1/driving/` +
        `${lng},${lat};${customerLng},${customerLat}` +
        `?overview=false`

      const res = await fetch(url)
      const data = await res.json()

      if (data.routes?.[0]) {
        const mins = Math.ceil(
          data.routes[0].duration / 60
        )

        setEta(
          mins < 60
            ? `${mins} min away`
            : `${Math.floor(mins/60)}h ${mins%60}m away`
        )
      }
    } catch {
      // ETA unavailable — silent fail
    }
  }

  return () => {

```

```

        socketRef.current?.disconnect()
    }
}, [bookingId, customerLat, customerLng])

return (
  <div style={{
    borderRadius : '16px',
    overflow      : 'hidden',
    border        : '1px solid rgba(10,255,230,0.2)',
  }}>
    { /* ETA Banner */ }
    <div style={{
      background    : cleanerPos
        ? '#0AFFE6' : '#161616',
      padding       : '14px 20px',
      display       : 'flex',
      alignItems    : 'center',
      justifyContent : 'space-between',
      transition    : 'background 0.5s',
    }}>
      <div style={{
        display      : 'flex',
        alignItems   : 'center',
        gap          : '10px'
      }}>
        <span style={{ fontSize:'24px' }}> 🚚 </span>
        <div>
          <p style={{
            fontWeight : '700',
            color       : cleanerPos ? '#000' : '#FFF',
            margin      : 0,

```

```

        fontSize : '15px',
    }}>
    {cleanerPos
      ? 'Your cleaner is on the way!'
      : 'Waiting for cleaner to start...'}
  </p>
  {eta && (
    <p style={{
      color : '#000',
      fontSize: '13px',
      margin : 0,
    }}>
      {eta}
    </p>
  )}
</div>
</div>

```

```

<div style={{
  display : 'flex',
  alignItems: 'center',
  gap : '6px',
  background: 'rgba(0,0,0,0.15)',
  padding : '4px 10px',
  borderRadius: '20px',
}}>
  <div style={{
    width : '8px',
    height : '8px',
    borderRadius: '50%',
    background : connected
  }}>

```

```

        ? '#22C55E' : '#EF4444',
    }}/>
    <span style={{
        fontSize: '11px',
        color : connected ? '#22C55E' : '#EF4444',
        fontWeight: '600',
    }}>
        {connected ? 'LIVE' : 'RECONNECTING'}
    </span>
</div>
</div>

```

```

{/* Map */}
<MapContainer
    center={[customerLat, customerLng]}
    zoom={13}
    style={{ height:'380px', width:'100%' }}
    zoomControl={true}
    scrollWheelZoom={true}
>
    <TileLayer
        url={DARK_TILES.url}
        attribution={DARK_TILES.attribution}
    />

```

```

{/* Auto-pan to cleaner */}
<MapPanner pos={cleanerPos}/>

```

```

{/* Customer home pin */}
<Marker
    position={[customerLat, customerLng]}

```

```

        icon={homePin}
      >
      <Popup>
        <strong>Your location</strong>
      </Popup>
    </Marker>

```

```

    {/* Cleaner moving pin */}
    {cleanerPos && (
      <Marker
        position={[cleanerPos.lat, cleanerPos.lng]}
        icon={cleanerPin}
      >
        <Popup>
          <strong>Your cleaner</strong>
          {eta && <><br/>{eta}</>}
        </Popup>
      </Marker>
    )}

```

```

    {/* Movement trail */}
    {trail.length > 1 && (
      <Polyline
        positions={trail.map(p => [p.lat, p.lng])}
        pathOptions={{
          color : '#0AFFE6',
          weight : 3,
          opacity : 0.5,
          dashArray: '8 4',
        }}
      />
    )}

```

```

    }}
  </MapContainer>

  { /* Footer */ }
  <div style={{
    background : '#0A0A0A',
    padding : '10px 16px',
    display : 'flex',
    justifyContent : 'space-between',
    alignItems : 'center',
  }}>
    <span style={{
      color : '#A0A0A0',
      fontSize: '12px',
    }}>
      ● Live — updates in real time
    </span>
    {lastUpdate && (
      <span style={{
        color : '#606060',
        fontSize: '11px',
      }}>
        Last update: {lastUpdate}
      </span>
    )}
  </div>
</div>
)
}

export default LiveTrackingMap

```

Show in booking detail when

status = ASSIGNED or IN_PROGRESS:

```
{{(booking.status === 'ASSIGNED' ||  
  booking.status === 'IN_PROGRESS') &&  
  booking.addressLat && (  
    <LiveTrackingMap  
      bookingId={booking.id}  
      customerLat={booking.addressLat}  
      customerLng={booking.addressLng}  
    />  
  )}}}
```

STEP 8 — Cleaner Location App

CREATE src/pages/CleanerApp.tsx

Route: /cleaner (add to App.tsx)

```
import { useState, useRef } from 'react'
```

```
import { io } from 'socket.io-client'
```

```
const CleanerApp = () => {  
  const [tracking, setTracking] = useState(false)  
  const [bookingId, setBookingId] = useState('')  
  const [status, setStatus] = useState('')  
  const watchIdRef = useRef<number | null>(null)  
  const socketRef = useRef<any>(null)
```

```

const startTracking = () => {
  if (!bookingId.startsWith('SC-')) {
    alert('Enter valid booking ID (SC-...)')
    return
  }

  if (!navigator.geolocation) {
    alert('GPS not supported on this device')
    return
  }

  socketRef.current = io(
    process.env.REACT_APP_API_URL ||
    'http://localhost:3001'
  )

  socketRef.current.on('connect', () => {
    setStatus('Connected )
    socketRef.current.emit(
      'cleaner:join', { bookingId }
    )
  })

  socketRef.current.on('disconnect', () => {
    setStatus('Reconnecting...')
  })

  watchIdRef.current = navigator.geolocation
    .watchPosition(
      (pos) => {
        const { latitude, longitude,

```

```

        heading, speed } = pos.coords

socketRef.current?.emit(
  'cleaner:location',
  { bookingId,
    lat: latitude,
    lng: longitude,
    heading, speed }
)

setStatus(
  `Sharing: ${latitude.toFixed(4)},
  ${longitude.toFixed(4)}`
)
},
(err) => {
  setStatus('GPS error: ' + err.message)
},
{
  enableHighAccuracy: true,
  maximumAge      : 3000,
  timeout         : 10000,
}
)

setTracking(true)
}

const stopTracking = () => {
  if (watchIdRef.current !== null) {
    navigator.geolocation
      .clearWatch(watchIdRef.current)
  }
}

```

```

    }

    socketRef.current?.disconnect()

    setTracking(false)

    setStatus('Stopped')
  }

  return (
    <div style={{
      minHeight    : '100vh',
      background    : '#0A0A0A',
      display       : 'flex',
      flexDirection : 'column',
      alignItems    : 'center',
      justifyContent : 'center',
      padding       : '24px',
      fontFamily    : 'Inter, sans-serif',
    }}>
      <div style={{
        background : '#161616',
        border     : '1px solid rgba(10,255,230,0.2)',
        borderRadius : '20px',
        padding    : '32px 24px',
        width      : '100%',
        maxWidth   : '360px',
        textAlign  : 'center',
      }}>
        <span style={{ fontSize:'48px' }}> 🍷 </span>

        <h1 style={{
          color    : '#FFFFFF',
          fontSize : '22px',

```

```
fontWeight: '700',
margin : '16px 0 4px',
}}>
SparkClean Staff
</h1>
```

```
<p style={{
  color : '#A0A0A0',
  fontSize : '14px',
  margin : '0 0 24px'
}}>
  Share your live location
</p>
```

```
{!tracking && (
  <input
    placeholder="Booking ID (SC-YYYYMMDD-XXXX)"
    value={bookingId}
    onChange={e =>
      setBookingId(e.target.value.toUpperCase())
    }
    style={{
      width : '100%',
      padding : '14px 16px',
      background : '#0A0A0A',
      border : '1px solid rgba(255,255,255,0.1)',
      borderRadius : '12px',
      color : '#FFFFFF',
      fontSize : '14px',
      marginBottom : '16px',
      outline : 'none',
```

```

        textAlign : 'center',
        boxSizing : 'border-box',
    }}
/>
})

```

```

{status && (
  <div style={{
    background : 'rgba(10,255,230,0.05)',
    border : '1px solid rgba(10,255,230,0.2)',
    borderRadius : '10px',
    padding : '10px 16px',
    marginBottom : '16px',
    fontSize : '12px',
    color : '#0AFFE6',
    wordBreak : 'break-all',
  }}>
    {status}
  </div>
)}

```

```

{!tracking ? (
  <button
    onClick={startTracking}
    disabled={!bookingId}
    style={{
      width : '100%',
      padding : '16px',
      background : bookingId
        ? '#0AFFE6' : '#333',
      color : bookingId

```

```
      ? '#000' : '#666',
    fontWeight : '700',
    fontSize   : '15px',
    borderRadius : '12px',
    border     : 'none',
    cursor     : 'pointer',
    transition : 'all 0.2s',
  }}
>
```

 Start Sharing Location

```
</button>
```

```
): (
```

```
<>
```

```
<div style={{
  display      : 'flex',
  alignItems   : 'center',
  justifyContent: 'center',
  gap          : '8px',
  marginBottom : '16px',
}}>
```

```
<div style={{
  width      : '100px',
  height     : '100px',
  background : '#22C55E',
  borderRadius : '50%',
}}/>
```

```
<span style={{
  color      : '#22C55E',
  fontSize   : '14px',
  fontWeight : '600',
```

```

    }}>
    Location sharing active
  </span>
</div>

<p style={{
  color : '#A0A0A0',
  fontSize : '13px',
  margin : '0 0 16px',
}}>
  Customer can see you on map
</p>

<button
  onClick={stopTracking}
  style={{
    width : '100%',
    padding : '14px',
    background : 'rgba(239,68,68,0.1)',
    color : '#EF4444',
    fontWeight : '600',
    fontSize : '15px',
    borderRadius : '12px',
    border : '1px solid #EF4444',
    cursor : 'pointer',
  }}
>
   Stop & Complete Job
</button>
</>
})

```

```
    </div>
  </div>
)
}
```

```
export default CleanerApp
```

STEP 9 — Backend Socket.io Setup

In src/index.ts (or server.ts)

REPLACE app.listen with httpServer:

```
import { createServer } from 'http'
import { Server } from 'socket.io'

const httpServer = createServer(app)

const io = new Server(httpServer, {
  cors: {
    origin : process.env.FRONTEND_URL,
    credentials: true,
    methods : ['GET', 'POST'],
  }
})

io.on('connection', (socket) => {
  console.log('Socket connected:', socket.id)

  socket.on('cleaner:join', ({ bookingId }) => {
```

```
socket.join(`booking:${bookingId}`)  
console.log(`Cleaner in room: ${bookingId}`)  
})
```

```
socket.on('cleaner:location', async ({  
  bookingId, lat, lng, heading, speed  
}) => {  
  try {  
    await prisma.cleanerLocation.upsert({  
      where : { bookingId },  
      update: { lat, lng, heading, speed },  
      create: {  
        bookingId,  
        cleanerId: 'unassigned',  
        lat, lng,  
        heading: heading || 0,  
        speed : speed  || 0,  
      }  
    })  
  } catch (err) {  
    console.error('Location save error:', err)  
  }  
}
```

```
io.to(`booking:${bookingId}`)  
  .emit('location:update', {  
    lat, lng, heading,  
    timestamp: new Date().toISOString()  
  })  
})
```

```
socket.on('customer:watch', ({ bookingId }) => {
```

```
socket.join(`booking:${bookingId}`)  
console.log(`Customer watching: ${bookingId}`)  
})
```

```
socket.on('disconnect', () => {  
  console.log('Socket disconnected:', socket.id)  
})  
})
```

```
httpServer.listen(  
  process.env.PORT || 3001,  
  () => console.log(  
    `Server + Socket.io on port ${  
      process.env.PORT || 3001  
    }`  
  )  
)
```

STEP 10 — Service Areas Map

CREATE src/components/ServiceAreaMap.tsx

Shows all cities on one map:

Live cities = teal circles

Coming soon = grey dashed circles

```
import { MapContainer, TileLayer,  
  Circle, Popup } from 'react-leaflet'  
import { DARK_TILES } from '../lib/mapConfig'
```

```
import { useNavigate } from 'react-router-dom'
```

```
const cities = [
```

```
{
  name : 'Bengaluru',
  state : 'Karnataka',
  lat : 12.9716,
  lng : 77.5946,
  status : 'LIVE',
  areas : ['Koramangala','Indiranagar',
    'Whitefield','HSR Layout'],
```

```
},
```

```
{
  name : 'Mumbai',
  state : 'Maharashtra',
  lat : 19.0760,
  lng : 72.8777,
  status : 'LIVE',
  areas : ['Bandra','Andheri',
    'Powai','Thane'],
```

```
},
```

```
{
  name : 'Visakhapatnam',
  state : 'Andhra Pradesh',
  lat : 17.6868,
  lng : 83.2185,
  status : 'COMING_SOON',
  areas : ['MVP Colony','Madhurawada'],
```

```
},
```

```
{
  name : 'Hyderabad',
```

```

    state : 'Telangana',
    lat   : 17.3850,
    lng   : 78.4867,
    status : 'COMING_SOON',
    areas : ['Banjara Hills','Gachibowli'],
  },
  {
    name   : 'Chennai',
    state  : 'Tamil Nadu',
    lat    : 13.0827,
    lng    : 80.2707,
    status : 'COMING_SOON',
    areas  : ['Anna Nagar','T. Nagar'],
  },
  {
    name   : 'Delhi',
    state  : 'NCR',
    lat    : 28.6139,
    lng    : 77.2090,
    status : 'COMING_SOON',
    areas  : ['Connaught Place','Gurugram'],
  },
]

```

```

const ServiceAreaMap = () => {
  const navigate = useNavigate()

```

```

  return (
    <div style={{
      borderRadius : '16px',
      overflow     : 'hidden',

```

```
border    : '1px solid rgba(10,255,230,0.2)',  
}}>
```

```
<MapContainer  
  center={[18, 78]}  
  zoom={5}  
  style={{ height:'420px', width:'100%' }}  
  zoomControl={true}  
  scrollWheelZoom={false}  
  attributionControl={false}
```

```
>
```

```
<TileLayer url={DARK_TILES.url}/>
```

```
{cities.map(city => (
```

```
  <Circle  
    key={city.name}  
    center={[city.lat, city.lng]}  
    radius={25000}  
    pathOptions={  
      city.status === 'LIVE'  
      ? {  
        color    : '#0AFFE6',  
        fillColor : '#0AFFE6',  
        fillOpacity: 0.15,  
        weight   : 2,  
      }  
    }  
  )
```

```
  : {  
    color    : '#555555',  
    fillColor : '#333333',  
    fillOpacity: 0.1,  
    weight   : 1,  
    dashArray : '8 4',
```

```

    }
  }
>
<Popup>
  <div style={{
    fontFamily: 'Inter, sans-serif',
    minWidth : '180px'
  }}>
    <p style={{
      fontWeight : '700',
      fontSize : '15px',
      margin : '0 0 4px',
      color : '#1A1A1A'
    }}>
      {city.name}
    </p>
    <p style={{
      color : '#666',
      fontSize : '12px',
      margin : '0 0 8px'
    }}>
      {city.state}
    </p>

    <span style={{
      background : city.status === 'LIVE'
        ? 'rgba(34,197,94,0.15)'
        : 'rgba(245,158,11,0.15)',
      color : city.status === 'LIVE'
        ? '#166534' : '#92400e',
      padding : '3px 10px',

```

```

borderRadius : '20px',
fontSize    : '11px',
fontWeight  : '600',
}}>
{city.status === 'LIVE'
  ? '✅ LIVE NOW'
  : '🔒 COMING SOON'}
</span>

```

```

<div style={{ marginTop:'10px' }}>
  {city.areas.map(a => (
    <span key={a} style={{
      display    : 'inline-block',
      background  : '#F5F5F5',
      color       : '#555',
      padding     : '2px 8px',
      borderRadius : '4px',
      fontSize    : '11px',
      margin      : '2px',
    }}>
      {a}
    </span>
  ))}
</div>

```

```

{city.status === 'LIVE' ? (
  <button
    onClick={() => navigate('/book')}
    style={{
      width      : '100%',
      marginTop  : '12px',

```

```

padding    : '8px',
background : '#0AFFE6',
color      : '#000',
fontWeight : '700',
fontSize   : '13px',
borderRadius : '8px',
border     : 'none',
cursor     : 'pointer',
}}
>
Book Now →
</button>
): (

```

```

href={`https://wa.me/919392420643?text=Notify me when SparkClean launches in
${city.name}`}

```

```

target="_blank"
rel="noreferrer"
style={{
  display    : 'block',
  width      : '100%',
  marginTop  : '12px',
  padding    : '8px',
  background  : 'transparent',
  color      : '#0AFFE6',
  fontWeight : '600',
  fontSize   : '13px',
  borderRadius : '8px',
  border     : '1px solid #0AFFE6',
  textAlign  : 'center',
  textDecoratation:'none',

```

```
    }}
  >
     Notify Me
  </a>
  )}
</div>
</Popup>
</Circle>
)})
</MapContainer>
```

```
<div style={{
  background : '#0A0A0A',
  padding    : '12px 20px',
  display    : 'flex',
  gap        : '20px',
  alignItems : 'center',
}}>
  <div style={{
    display : 'flex',
    alignItems: 'center',
    gap      : '6px'
  }}>
    <div style={{
      width      : '12px',
      height     : '12px',
      borderRadius : '50%',
      background  : '#0AFFE6',
    }}/>
    <span style={{
      color : '#A0A0A0',
```

```

        fontSize: '12px'
      }}>
      Live now
    </span>
  </div>
  <div style={{
    display : 'flex',
    alignItems: 'center',
    gap    : '6px'
  }}>
    <div style={{
      width    : '12px',
      height   : '12px',
      borderRadius : '50%',
      background : '#555',
    }}/>
    <span style={{
      color : '#A0A0A0',
      fontSize: '12px'
    }}>
      Coming soon
    </span>
  </div>
</div>
)
}

```

export default ServiceAreaMap

Replace existing ServiceAreas pills section

with `<ServiceAreaMap />` component.

STEP 11 — Prisma Schema Update

In `prisma/schema.prisma`

ADD to Booking model:

`addressLat Float?`

`addressLng Float?`

ADD new model:

```
model CleanerLocation {  
  id      String  @id @default(cuid())  
  bookingId String  @unique  
  cleanerId String  
  lat     Float  
  lng     Float  
  heading Float?  @default(0)  
  speed   Float?  @default(0)  
  updatedAt DateTime @updatedAt  
  createdAt DateTime @default(now())  
}
```

Run:

`npx prisma migrate dev --name add_maps_location`

`npx prisma generate`

STEP 12 — App.tsx Routes

ADD to App.tsx:

```
import CleanerApp from './pages/CleanerApp'
```

```
<Route path="/cleaner" element={<CleanerApp/>}/>
```

STEP 13 — Deploy

```
git add .
```

```
git commit -m "feat: maps + live tracking (Leaflet)"
```

```
git push origin main
```

Vercel auto deploys frontend 

Render auto deploys backend 

Test:

1. Book a service

→ address autocomplete works 

2. Success page

→ map shows pin at address 

3. Go to /cleaner

→ Enter booking ID

→ Click Start Sharing 

4. View booking detail

→ Live map shows cleaner moving 

5. Service areas section

→ Interactive map with cities 

DO NOT CHANGE

- All existing pages
- Booking flow & payment
- Auth system
- Admin dashboard
- All colors and fonts
- Contact: 9392420643

Build everything now.

Zero API keys needed.

Total cost: ₹0 forever.